Multi-Agent Frameworks: Sample Intelligence Brief

Status: Staged sample (v3, November 7, 2026). Not certified. The accompanying evidence table maps retained factual passages to captured sources; checks were selective, and matching an excerpt is not verification of every implication. This is a replacement candidate built from the legacy public sample's structure and from the local October 22, 2026 research notes; it is not a corrected copy of the legacy PDF. Production-share, download, workflow-volume and price figures were removed rather than replaced. **Tags:** [Vendor] = the project's own description; [Source] = stated in the cited page; [Interpretation] = our judgment; [Proposed test] = a pilot step we recommend, not a measured result.

1. Direct Answer

This review supplies no independent, audited production-share estimate and makes no adoption ranking. The decision-relevant evidence it does supply is lifecycle status, documented capabilities, and a pilot method for comparing candidates on your own work.

- **AutoGen:** its README labels it archived and no longer maintained, advises against production use, and directs new users to the Microsoft Agent Framework [1]. **[Source]**
- **Microsoft Agent Framework (MAF):** Microsoft announced version 1.0 for .NET and Python on April 3, 2026 and calls it "the production-ready release" [2]. **[Vendor]**
- **OpenAI Swarm:** its README says it "is now replaced by the OpenAI Agents SDK" and recommends migrating for all production use cases [3]. **[Source]**
- **Other candidates:** LangGraph, CrewAI, the OpenAI Agents SDK, Google ADK, AWS Strands and Pydantic AI. This review does not establish their comparative suitability or any mandatory cloud dependence.

Bottom line [Interpretation]: do not start new work on AutoGen or Swarm. Shortlist two candidates whose documented capabilities match your workflow, and decide with the pilot in Section 5 rather than on popularity figures.

2. Key Evidence

2.1 Lifecycle status

Project	What the cited source says	Source
AutoGen	"Project archived and no longer maintained"; advises against production use; new users pointed to MAF. Its contribution note still says it "is in maintenance mode," with contributions limited to bug fixes, security patches and documentation.	[1]

Project	What the cited source says	Source
Microsoft Agent Framework	1.0 for .NET and Python, April 3, 2026; described by Microsoft as production-ready, with multi-provider model support, A2A and MCP interoperability, and migration assistants for Semantic Kernel and AutoGen users	[2][6]
OpenAI Swarm	Replaced by the OpenAI Agents SDK; migrate for production	[3]
Others	Lifecycle status not assessed in this review	n/a

2.2 Documented capabilities and evaluation profile

The table records what each cited source documents and what a pilot should test. The "workload to test" and "operational question" columns are our proposals, not measured comparisons.

Framework	Documented in cited source	Workload to test [Interpretation]	Operational question to answer [Proposed test]
LangGraph	Fetched documentation lists "Persistence" and "Fault tolerance" topics [4]. This review did not test recovery behaviour.	Long-running, branching or approval-gated workflows	Does a run resume correctly after a forced restart? Who owns state storage and retention?
CrewAI	Crews give language models "defined roles and goals"; Flows are described as "a thin, low-level layer with full control for advanced orchestration"; tracing ships with every execution [5].	Role-based business workflows	Do Crews plus Flows give the execution control your workflow needs? This review did not compare control or development speed.
MAF	Multi-agent orchestration, multi-provider models, A2A and MCP; official AutoGen migration guide [2][6].	Teams with existing AutoGen or Semantic Kernel code	How much code changes under the migration guide, and do behaviours match after migration?
OpenAI Agents SDK	"Provider-agnostic"; agents with tools, guardrails and handoffs; human-in-the-loop mechanisms; sessions; built-in tracing [7].	Handoff-heavy workflows with guardrails	Do guardrails and human approval steps fire where expected? Do traces explain each handoff?
Google ADK, AWS Strands, Pydantic AI	Not assessed here. The linked hosting and observability pricing pages [8][9][10] are for cost checks on your decision date only.	To be defined after a separate capability review	n/a

2.3 Audit questions for popularity signals [Proposed test]

This sample cites no download, star or volume figures. If a vendor or analyst offers one, ask:

- Is this a download, install, repository star or production deployment count? Do not treat any of these as interchangeable.
- What is the counting method? Does it include automated or dependency-driven downloads?
- What is the unit and time window, and is the figure independently verified?
- For vendor "execution" or "workflow" volumes: what counts as one execution, and over what period?

3. Strongest Counterpoints

1. **Framework architecture can materially affect performance.** MAFBench reports that "framework-level design choices alone" produce large differences in latency, planning and coordination under controlled model and task conditions. These are preprint findings, specific to its workloads and configurations, and are not production rankings [11].
2. **"Production-ready" is a vendor label.** MAF's 1.0 announcement does not demonstrate adoption or deployment quality [2].
3. **Control has an operational cost [Interpretation].** Persistent state, retention policies and tracing add infrastructure someone must own.
4. **Prototypes become production [Interpretation].** A framework chosen for demo speed can end up running critical paths without a planned migration.
5. **Custom orchestration remains an option [Interpretation].** A thin in-house control loop may suit narrow, stable workflows; its prevalence was not established in this review.

4. Residual Uncertainties

- Production deployment counts and reliability for every framework listed.
- Lifecycle status and capabilities of Google ADK, AWS Strands and Pydantic AI (not assessed).
- How quickly former AutoGen users adopt MAF, and how complete the migration path is in practice.
- Current pricing for hosted and observability tiers; check vendor pages on your decision date [8][9][10][12][13].
- Whether MAFBench's architecture effects hold on your workload [11].
- API compatibility over time for any candidate you pilot.

5. Actionable Recommendations: Pilot and Migration Playbook

These are recommended tests, not claims that every framework supports each feature.

1. **Stop new AutoGen and Swarm work.** Follow the official migration paths [1][3][6]. Before migrating, record the current system's behaviour on a fixed task set so you can compare afterwards.
2. **Pick two finalists from Section 2.2.** Treat the shortlist as untested pilot hypotheses, not comparative findings.
3. **Run a two-week matched-task pilot [Proposed test].** Use 20–50 real tasks, the same model and the same prompts for both finalists. Hold model version, tool access and budgets constant, or disclose unavoidable differences. Repeat runs to see variability. Record task success, latency, token use, cost per run and cost per successful task.
4. **Test failure and recovery [Proposed test].** Run this and the approval tests in a sandbox or with stubbed consequential tools, never against live irreversible actions. Kill a run mid-task and restart it. Record whether state, pending approvals and tool calls resume correctly, and how long recovery takes.
5. **Test approval boundaries [Proposed test].** Put a human approval step in front of every irreversible action. Attempt to bypass it in defined adversarial cases, and check that rejected actions are not executed in those cases. Record coverage and unresolved bypass risks. Passing these tests does not prove the boundary cannot be bypassed.
6. **Inspect traces [Proposed test].** For up to five failed tasks per finalist (report how many were available), check whether the traces let an engineer explain the failure without re-running it.
7. **Fill a cost worksheet on the decision date.** Include framework licence, observability, model API, hosting, storage and engineering time, using current vendor pricing pages [8][9][10][12][13].
8. **Set rollback criteria in advance.** For example, revert if success rate falls below the incumbent's, if cost per task rises beyond an agreed limit, or if recovery fails in any test. Write the thresholds down before the pilot starts.

5.1 Pilot acceptance sheet [Proposed test]

Write these criteria down before the pilot starts. Thresholds are yours to set; the sheet below defines what to record and how to judge it. It does not report results.

Test	What to record	Pass if	Fail or escalate if
Matched tasks	Success per task, latency, tokens, cost per successful task, run-to-run variance	Meets the success and cost thresholds you set in advance	Below the incumbent, or variance too high to compare

Test	What to record	Pass if	Fail or escalate if
Failure and recovery	State after restart, pending approvals, duplicated or lost tool calls, time to recover	Every forced failure resumes without lost or repeated consequential steps	Any lost approval, duplicate action or manual repair
Approval boundary	Bypass cases attempted, outcome for each, cases not tested	No rejected action executed in any tested case	Any bypass in a tested case; treat untested paths as open risk
Traces	Whether an engineer can explain each inspected failure from the trace alone	Most inspected failures explained without a re-run	Failures that need a re-run or code reading to explain
Migration (AutoGen or Swarm)	Behaviour on the fixed task set before and after migration	Differences are understood and accepted	Unexplained behaviour changes
Cost worksheet	Licence, observability, model API, hosting, storage, engineering time on the decision date	Total within budget at expected volume	Missing cost lines, or prices not checked on the decision date

Rollback rule: decide before the pilot which result in the "Fail or escalate" column returns you to the incumbent or the other finalist, and who signs off. A pilot pass is evidence for your workload only, not a general ranking of frameworks.

Sources

1. Microsoft AutoGen repository README. https://github.com/microsoft/autogen
2. Microsoft, "Microsoft Agent Framework Version 1.0" (Apr 3, 2026). https://devblogs.microsoft.com/agent-framework/microsoft-agent-framework-version-1-0/
3. OpenAI Swarm repository README. https://github.com/openai/swarm
4. LangGraph documentation (case-studies page and documentation navigation). https://docs.langchain.com/oss/python/langgraph/case-studies
5. CrewAI, "CrewAI OSS 1.0" announcement (Oct 2025). https://crewai.com/blog/crewai-oss-1-0---we-are-going-ga
6. Microsoft, "AutoGen to Microsoft Agent Framework Migration Guide." https://learn.microsoft.com/en-us/agent-framework/migration-guide/from-autogen/
7. OpenAI Agents SDK repository README. https://github.com/openai/openai-agents-python
8. Google Cloud, Gemini Enterprise Agent Platform pricing. https://cloud.google.com/products/gemini-enterprise-agent-platform/pricing
9. AWS, Amazon Bedrock AgentCore pricing. https://aws.amazon.com/bedrock/agentcore/pricing/
10. Pydantic Logfire observability pricing. https://pydantic.dev/pricing
11. MAFBench preprint (arXiv 2602.03128, Feb 2026). https://arxiv.org/html/2602.03128v1
12. LangSmith pricing. https://www.langchain.com/pricing-langsmith
13. CrewAI pricing. https://crewai.com/pricing